

DOFBOT × Xinu メッシュ 12 台 システム全体設計

2026 年 7 月 21 日

Abstract

6 軸アーム DOFBOT を、12 台の Xinu メッシュと強化学習で「次第にうまく動作する」システムとして構成するための設計。結論は 2 つ。**学習は Mac、推論は Xinu メッシュ**に置く (actor-learner 分離)。そして**メッシュを渡る周期は 10 Hz を超えてはならない** — 本日実測した往復レイテンシが中央値 15–130 ms、裾が秒単位だからである。本書はこの制約を出発点に、4 層のアーキテクチャ、データ経路、段階的な導入計画、そして着手前に潰すべき前提条件を示す。

1 設計の出発点 — 実測値

設計判断はすべて 2026-07-21 に実機 3 台で測った次の値に基づく。全ボード同一エンドポイント (/api/actors)、Mac から 200 回、10 ms 間隔。

ボード	中央値	p95	p99	最大	失敗率
Pi 3 (BCM2837)	15.3 ms	2022 ms	3009 ms	3012 ms	14 %
Pi 5 (BCM2712)	36.3 ms	38.4 ms	84.3 ms	319 ms	0 %
Pi 4 (BCM2711)	130.3 ms	166 ms	183 ms	543 ms	0 %

この表から設計上の制約が 3 つ導かれる。

- メッシュをサーボ制御ループに入れてはならない**。1 kHz ループのジッタ許容 0.1 ms に対し、最良の中央値でも 15 ms、裾は秒単位。
- メッシュを渡る周期は 10 Hz 以下**。p99 が 183 ms (Pi 4) あるため、100 ms 周期でも取りこぼす。ノード側で内挿する前提で 10 Hz とする。
- 通信は欠落前提で設計する**。Pi 3 は 14 % が失敗した。要求-応答ではなく、最新値が上書きされるデータグラム型が適する。

なお Pi 3 の失敗と 2 秒の裾は原因が特定できている (単一 accept の HTTP サーバに要求が重なると詰まる)。修正はコミット済みで未デプロイ。Pi 4 の 130 ms は **原因未究明**であり、有線接続で Pi 5 の 3.6 倍という異常値である。

2 結論: 学習は Mac、推論はメッシュ

強化学習を 3 つの部分に分けて、それぞれ置き場所を決める。

部分	置き場所	理由
方策の実行 (推論)	Xinu メッシュ	TinyML は 192→12→3 で約 2.4k 積和。ボードで十分回る。ロボットの制御経路上にあるので、ここがメッシュの本来の持ち場
方策の更新 (学習)	Mac	Xinu は D-cache が 3 ボードとも OFF (アセスメント項目 10、最大の性能レバーが未使用)、ベクトル化も数値ライブラリも無い。12 台 48 コアを足しても素の Mac に勝てる見込みが薄い
経験の収集	Xinu メッシュ	各ノードが自分の (s, a, r) を貯める。分散して当然の処理

これは強化学習では標準的な **actor-learner 分離** (Ape-X / IMPALA と同型) である。分散したアクターが経験を集め、中央の学習器が方策を更新して配り直す。既存資産とも噛み合う — 方策の配布には既にある **/actor/load** (AIPL → C JIT) がそのまま使える。

追加の **Xinu** を学習に使わない理由を明確にしておく。使えないのではなく、現状では割に合わない。D-cache を有効化し (項目 10)、数値カーネルを整備すれば状況は変わる。その時点で再評価すべき設計判断であり、恒久的な結論ではない。

3 アーキテクチャ

3.1 4 層構成

層	周期	実体	役割
L0 駆動	1–10 kHz	アーム内蔵 Pi 5 (Linux/ROS)	電流・位置ループ。Arm_Lib が I2C でバスサーボを叩く。 メッシュに載せない
L1 関節エージェント	10–100 Hz	Xinu 12 台	関節ごとに ServoMotor + ServoSensor。目標値の生成と状態推定、TinyML 推論。L0 へは設定値のみ送る
L2 協調	10 Hz	Xinu 1 台または Mac	タスク列、逆運動学、フェーズ遷移。CG シミュレータの Planner に相当
L3 学習	分オーダー	Mac	CEM/RL、データ集約、方策のコンパイルと配布

12 台の割当は、シミュレータが既に採っている「各サーボに 2 台 = ServoMotor + ServoSensor」をそのまま実機に写す。6 関節 $\times 2 = 12$ で過不足がない。

3.2 データ経路

経路	向き	周期	内容
設定値	L2 → L1 → L0	10 Hz	関節目標角。ノード側で内挿して L0 へ渡す
状態	L0 → L1 → L2	10 Hz	実測角、把持状態、カメラ特徴量
経験	L1 → L3	数秒に 1 回	(s, a, r) の束。欠落しても学習が壊れない粒度
方策	L3 → L1	学習の区切りごと	TinyML の重み。/actor/load で投入

設定値と状態は「最新値の上書き」であって「要求–応答」ではない。14% が失敗する経路の上で信頼性を積み上げるより、落ちたら次の値で上書きされる設計にする方が堅い。したがって L1–L2 間は **HTTP** ではなく **UDP** (MANET の HELLO が既に使っている UDP:5000 と同じ層) とする。

4 タイミング予算

10 Hz (100 ms) 周期に対する内訳。実測の p99 を使う。

項目	時間	根拠
L2 → L1 伝送	40 ms	Pi 5 実測 p99 84 ms の半分 (片道)
L1 推論 (TinyML)	<1 ms	2.4k 積和
L1 → L0 伝送	未測定	アーム内蔵 Pi との接続方式が未確定
L0 サーボ応答	未測定	カタログに速度の記載が無い
余裕	—	残り

下 2 行が空欄であることが、現時点の設計の最大の弱点である。 実機が来るまで埋まらない。したがって当面はこの 2 つをパラメータとして残し、感度解析で扱う。カタログに無い数字を「あることにする」よりも、結論がその数字にどう依存するかを示す方が、実機が来たときに測って差し替えるだけで済む。

5 段階的な導入

実機は順次購入して繋いでいく前提なので、台数ごとに何が成立するかを定める。

段階	台数	成立すること
現在	3	メッシュ疎通、レイテンシ実測、アクター実行。 制御は不可
第 1	3	CG シミュレータを実機メッシュに接続。ServoMotor 6 個を 3 台に分散し、CG が実機の出す指令で動く。 ここで初めて「Xinu が関節を動かす」
第 2	6	ServoMotor 6 個を 6 台に 1:1 割当。関節ごとの独立性が出る
第 3	12	ServoSensor 6 個を追加。設計どおりの 12 ノード構成
第 4	12 + 実機	L0 をアーム内蔵 Pi に接続。CG と実機の二重化

第 1 段階は今の 3 台で実施できる。CG シミュレータ (:8022) が実機ノードの /actor/send を叩き、返ってきた関節角で描画する形にすれば、新しいハードウェアなしに「実機 Xinu が CG のアームを動かす」ところまで行ける。そこで取れるサイクル時間とジッタが、12 台に増やす価値を判断する実数になる。

6 着手前に潰すべき前提

設計そのものより先に、次が片付いていないと数字が信用できない。

1. **Pi 3 の 14% 失敗**。原因は特定済み（単一 accept の HTTP サーバ）、修正はコミット済み・未デプロイ。**デプロイして効果を実測する**。
2. **Pi 4 の 130 ms**。有線で Pi 5 の 3.6 倍。原因未究明。メッシュ全体の応答を律速している。
3. **時刻同期が無い**。各ノードが自分の時計を見ているだけで、ノード間の共通時刻がない。経験 (s, a, r) をノード横断で突き合わせるには必須。Pi 3 は時間軸自体が壊れていた（本日修正、clkticks は秒内カウンタで非単調、かつ 100 Hz 前提で 10 倍ずれ）。
4. **D-cache が 3 ボードとも OFF**。学習をメッシュに載せるか否かの判断は、これを有効化してから再評価する。
5. **サーボ速度がカタログに無い**。RL の結論が依存する。感度解析で扱い、実機で測って差し替える。

7 本書の限界

- ・レイテンシは 200 サンプル × 3 台、1 回の測定である。時間帯や電波状況による変動は評価していない。
- ・L1-L0 間とサーボ応答は**未測定**であり、タイミング予算に空欄がある。
- ・12 台構成の効果（4-6 倍）は**外挿**であり、実測は 3 台 1.87 倍のみ。
- ・アーム内蔵 Pi 5 に ROS が載っているという前提は**未確認**。製品構成からの推定であり、実機が来るまで検証できない。

実測は Raspberry Pi 3 B+ / 4 / 5 上の Xinu、MANET は WiFi ad-hoc (IBSS)、SSID MANET、ch 6、10.0.0.2/3/5。CG シミュレータは ~/aipl_line_simulator (:8022)。